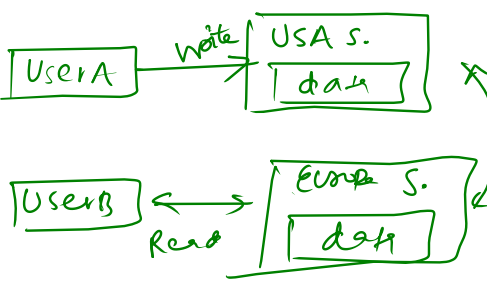


CAP theorem

You can only have 2 out of 3:

- 1. Consistency: → all nodes/users see the same data at the same time
- 2. Avail: every Req get a Response (Successful or not)
- 3. partition tolerance: - sys. works despite N/w failures b/w nodes

Must



1. Requirements: ↓ think

Consistency of Availability ⇒ which one to forfeit

Need strong consistency.

- ① Ticket Booking (airline, event etc).
- ② Inventory system
- ③ Financial systems

If you don't need strong consistency choose Availability: →

- ① Social media app (posts, photos)
- ② Yelp like business Reviews
- ③ Streaming services like Netflix

How does it influence my design?

Strong consistency -

- ① Implement distributed transactions
- ② Limit to a single Node
- ③ discuss consensus protocols
- ④ accept higher latency

Example tools: →

PostgreSQL
Trade & Dmes
Spinner
NO SQL Strong Consistency (dynamic)

- N/w fails b/w them
- 1. STOP serving data (strong consistency)
 - 2. OR Replicate data (availability)

diff parts of a system can have diff Req.

TicketMaster → Avail for CRUD
→ consistency for booking tickets

Tinder → Avail for view profile
→ consistency for matches

Availability

→ use multiple Replicas

→ CDC and eventual consistency b/w

diff types of consistency

- 1. Strong consistency: - all Reads reflect most recent Write. Example: PostgreSQL
- 2. Causal consistency: → Related events app. in order
- 3. Read your writes consistency: User see their own updates
- 4. Eventual consistency: updates will propagate eventually

~~XXXXXX~~

